GeoFare Mobile Application

Error Exception Analysis

Software Engineering Principle: Error Exception / Exception Handling

Project technology: Kotlin, Jetpack Compose, Firebase Authentication, and Cloud Firestore **Basis:**

Analysis of the GeoFare source code supplied by the developer.

Purpose of this document

This document evaluates how the submitted GeoFare source code handles errors and exceptions. The assessment focuses on the exception-handling concepts discussed in the lecture, including **try**, **catch**, **throw**, and **finally**, while also recognizing Firebase's asynchronous success and failure callbacks as practical errorhandling mechanisms.

1. Summary of Findings

Area	Finding	Status
try block	No try-catch block appears in the supplied files.	Not implemented
catch block	No catch clause appears in the supplied files.	Not implemented
throw	No explicit throw statement appears in the supplied files.	Not implemented
finally	No finally block appears in the supplied files.	Not implemented
Firebase failure handling	Login and registration use task/failure callbacks to handle operation failures.	Implemented
User error messages	The UI displays errorMessage values for several failures.	Implemented

2. Primary Evidence — LoginScreen.kt

The strongest evidence of error handling is in **LoginScreen.kt**. Firebase Authentication reports whether the login operation succeeds. When it fails, the code checks **task.exception** and maps known Firebase exceptions to user-friendly messages.

```
auth.signInWithEmailAndPassword(email, password)
.addOnCompleteListener { task ->         isLoading = false
if (task.isSuccessful) {
navController.navigate("home") {
popUpTo("login") { inclusive = true }             }         }
else {             errorMessage = when (task.exception) {
is FirebaseAuthInvalidUserException ->                     "No
account found with this email"                is
FirebaseAuthInvalidCredentialsException ->
"Invalid password"                else ->
"Login failed: ${task.exception?.message}"
            }
        }
}
```

Explanation: The code does not use a traditional try-catch structure, but it does handle a failed asynchronous Firebase operation. It identifies specific exception types—**FirebaseAuthInvalidUserException** and **FirebaseAuthInvalidCredentialsException**—and gives the user an appropriate error message. This is valid practical error handling, although it is different from Kotlin's explicit try-catch exception handling.

3. Registration Error Handling — RegisterScreen.kt

RegisterScreen.kt also contains error handling for Firebase registration and Firestore operations. The code uses **addOnFailureListener** when saving user data and checks **task.isSuccessful** after account creation.

```
firestore.collection("users")
.document(userId)
    .set(user)
    .addOnSuccessListener      {
isLoading        =        false
navController.navigate("home") {
            popUpTo("register") { inclusive = true }
}
```

```
    }     .addOnFailureListener {         isLoading
= false         errorMessage = "Failed to save
user data"     }
```

Explanation: If Firestore cannot save the user data, the failure listener changes the loading state and displays an error message instead of silently ignoring the failure.

4. Additional Registration Error Handling

```
if (task.isSuccessful) {     ... } else {     isLoading =
false     errorMessage =         "Registration failed:
${task.exception?.message}"
}
```

This checks the result of Firebase account creation. If the operation fails, the application stores an error message for display to the user. The registration screen also validates required fields, email format, password length, and password confirmation before calling Firebase.

5. What Is NOT Implemented

Based only on the source files supplied, the project does not contain an explicit Kotlin **try-catch-finally** structure. There is also no explicit **throw** statement. Therefore, if your instructor specifically requires the lecture's traditional exception syntax, GeoFare currently needs improvement in this area.

Missing example:

```
try {     // operation that may throw an
exception
} catch (e: Exception) {
    // handle the exception
} finally {
    // optional cleanup
}
```

The example above is a recommendation only; it is not claimed to be existing GeoFare code.

6. Recommended Improvement

Where an operation can actually throw a Kotlin/Java exception, the application can use a targeted try-catch structure and handle only the exceptions that are expected. Broadly catching every exception should be avoided when a more specific exception type is available. Firebase's existing success/failure listeners should also be retained because they are appropriate for asynchronous Firebase operations.

7. Conclusion

The supplied GeoFare code demonstrates practical error handling, particularly in LoginScreen.kt and RegisterScreen.kt, through Firebase task results, exception-type checks, failure listeners, and user-facing error messages. However, the supplied files do not demonstrate explicit try, catch, throw, or finally blocks. Therefore, the most accurate conclusion is that **error handling is implemented, while traditional explicit exception-handling syntax is not implemented in the reviewed source code**.